



Administrator Linux.Professional

Управление пакетами. Дистрибьюция софта



Проверить, идет ли запись

Меня хорошо видно && слышно?



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Телеграм-чате



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

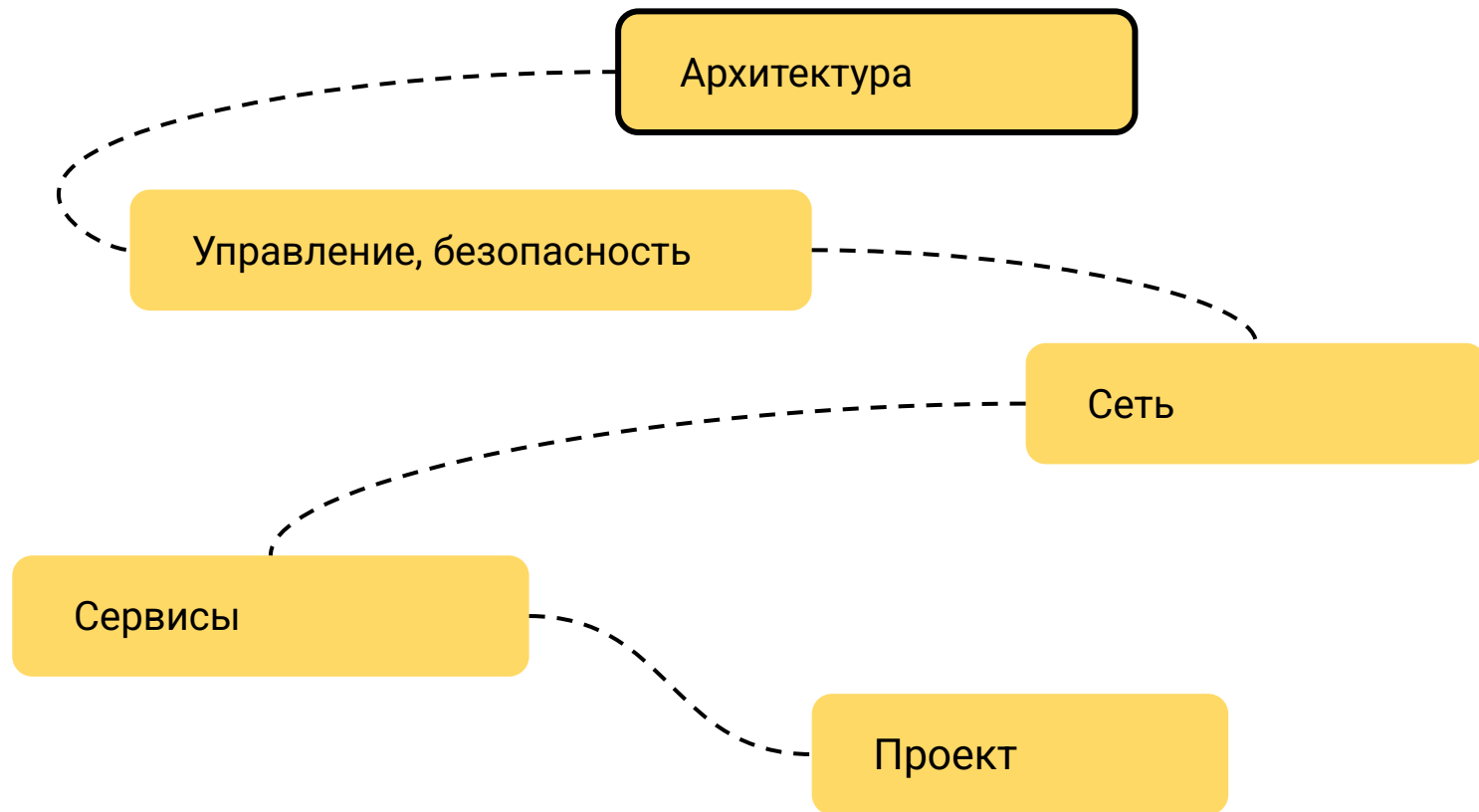


Документ

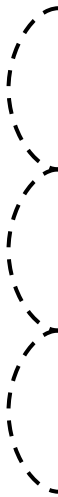


Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



Пакетные менеджеры

Классические пакеты

Автономные пакеты

Сборка DEB и RPM-пакетов

Цели вебинара

После занятия вы сможете

1. Понимать принципы управления пакетами
2. Узнать процесс сборки пакетов

Смысл

Зачем вам это уметь

1. Работать с пакетами
2. Собирать собственные версии пакетов



Сборка софта из ИСХОДНЫХ КОДОВ

Процесс сборки из исходных кодов

- Получение исходных кодов (wget)
 - Архив с кодом приложения
- Конфигурация (опции) — `./configure`
 - Настройка сборки
 - Включение/выключение компонентов
 - Подключение библиотек
- Компиляция программы из исходных кодов — `make`
 - Формирование бинарных файлов
- Установка — `make install`
 - Копирование собранных файлов

Причины сборки

- Нет готового пакета
- Эксперименты с новейшими версиями
- Участие в разработке
- Отладка, поиск проблем
- Требуется индивидуальный вариант сборки (модули, опции)

Пример сборки Nginx

```
yum groupinstall "Development Tools" -y
yum install kernel-devel kernel-headers -y
mkdir nginx && cd nginx
wget https://nginx.org/download/nginx-1.19.5.tar.gz
tar xvf nginx-1.19.5.tar.gz
cd nginx-1.19.5
./configure
yum install pcre pcre-devel -y
yum install zlib zlib-devel -y
make
make install
```

Недостатки ручной сборки

- Требуется наличия сборочного окружения на сервере
- Занимает много времени
- Требуется наличия исходного кода
- Ручное разрешение зависимостей
- Сложность тиражирования
- Возможны затруднения с systemd unit и init-скриптами
- Больше затрат на поддержку системы (обновления безопасности)
- Общее стремление к хаосу в системе

Пакетные менеджеры



Какие знаете?

Пакетные менеджеры

- **Классические пакеты (система зависимостей)**
 - **RPM Package Manager** – Red Hat (.rpm)
 - **DPKG** – Debian (.deb)
 - **Pacman** – ArchLinux (.pkg.tar.zst)
- **Автономные пакеты (независимые)**
 - **Snap** – Ubuntu
 - **Flatpack**
 - **AppImage**

Задачи пакетных менеджеров

- Упаковка ПО
- Быстрая установка
- Избавление пользователя от сборки
- Централизация управления
- Автоматизация
- Частые обновления
- Разрешение зависимостей
- Формирование дистрибутивов

Репозитории (repository)

- Файловое хранилище пакетов
- Списки пакетов (индексы)
- Могут иметь несколько разделов
- Пакеты обычно подписаны с помощью ключа
- Одновременно только одна версия пакета в системе
- Пример: <http://ru.archive.ubuntu.com/ubuntu/>

Создание пакета

- Получение исходного кода (от upstream — разработчика ПО)
- У пакета может быть maintainer — автор сборки
- Наложение патчей
- Добавление инфраструктуры (конфиги, systemd units)
- Скрипты установки и удаления
- Указание зависимостей, конфликтов
- Сборка пакета в контексте дистрибутива
- Тестирование
- Версионирование двойное: upstream + build
- Название пакета: `bash_5.2.15-2ubuntu1_amd64.deb`

Классификация репозиториев

- Происхождение
 - Разработчик дистрибутива
 - Разработчик ПО
- Лицензия
 - Открытое ПО (open-source software)
 - Закрытое ПО (proprietary, binary blobs)
- Тип пакетов
 - Дистрибутив
 - Обновления
 - Исходники
 - Бэкпорты

Классические пакеты



RPM

- Используется в Red Hat дистрибутивах
- Форматы: *.rpm (бинарные), *.src.rpm (исходники)
- Базовая утилита: rpm
- Управление с зависимостями: yum, dnf
- Конфигурация в CentOS: /etc/yum, /etc/yum.repos.d

Работа с RPM-пакетами

- `rpm -ivh <file>` - установка из файла
- `yum (dnf) install <package>` - установка из репозитория
- `yum localinstall <file>` - установка из файла
- `rpm -Uvh <file>` - обновление из файла
- `rpm -ev <package>` - удаление пакета
- `yum (dnf) remove <package>` - удаление пакета
- `yum update` - обновление всех пакетов
- `dnf upgrade` - обновление всех пакетов

Работа с RPM-пакетами

- `yum (dnf) search` – поиск пакета
- `rpm -qi <package>` – информация о пакете
- `rpm -ql <package>` – список файлов пакета
- `rpm -qa` – список установленных пакетов
- `yum makecache` – обновить список пакетов
- `yum check-update` – проверить наличие обновлений

DPKG (DEB)

- Используется в Debian дистрибутивах
- Форматы: *.deb (бинарные), *.tar.xz (исходники)
- Базовая утилита: dpkg
- Управление с зависимостями: apt, apt-get, aptitude
- Конфигурация в Ubuntu: /etc/apt, /etc/apt/sources.list.d

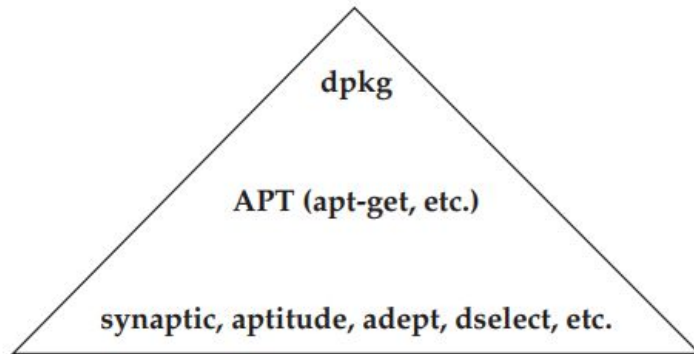
Работа с DEB-пакетами

- `dpkg -i <file>` – установка из файла
- `apt install <package|file>` – установка из файла, репозитория
- `dpkg -r <package>` – удаление пакета
- `apt remove <package>` – удаление пакета
- `apt purge <package>` – удаление пакета целиком
- `apt update` – обновление списка пакетов
- `apt upgrade` – обновление всех пакетов



Работа с DEB-пакетами

- `apt -f install` – разрешение проблем с пакетами
- `apt autoremove` – удаление ненужных пакетов
- `apt clean` – удалить скачанные пакеты
- `apt list <pattern>` – список пакетов по названию
- `apt-cache showpkg <package>` – информация о пакете
- `dpkg -L <package>` – список установленных файлов



Автономные пакеты



Автономные пакеты

- Примеры: Snap, Flatpack, AppImage
- Решают проблемы совместимости версий дистрибутивов
- Упрощают(?) жизнь разработчикам
 - приносят другие проблемы
- Могут повышать безопасность (изоляция)
- Больше занимают места
- Меньше эффективность использования ресурсов
- Нет универсального формата
- Разнообразие графических оболочек ломает систему
- Хороший вариант при отсутствии стандартных пакетов



Работа со snap-пакетами

- `snap search <pattern>` – поиск пакетов
- `snap install <package>` – установка пакета
- `snap refresh` – обновление пакетов
- `snap remove` – удаление пакета

Сборка RPM-пакетов



Процесс пересборки RPM-пакета

- Загрузим `src.rpm` файл с исходниками
- Установим зависимости
- Соберем пакет
- Установим и проверим работу сервиса

Сборка RPM-пакета с нуля

- Создаем директорию для сборки
- Получаем исходный код
- Пишем spec-файл
(<https://github.com/nixuser/package-example/blob/main/shellspec.spec>)
- Собираем пакет
- Устанавливаем пакет

Сборка RPM-пакета с нуля

```
yum install -y rpmdevtools rpm-build
```

```
rpmdev-setuptree # создаем дерево
```

```
tree -d -L 1 ~/rpmbuild
```

```
/home/vagrant/rpmbuild
```

|— BUILD # здесь происходит сборка из SOURCES

|— BUILDROOT # здесь установлены файлы

|— RPMS

|— SOURCES # здесь исходные файлы

|— SPECS

|— SRPMS



Секции спес-файла

- The Preamble
- The %prep Section. (sh script)
- The %build Section. (sh script)
- The %install Section (sh script)
- The %files Section (list of the files that are part of the package)
- The %post %postun Install/Uninstall Scripts
- The %clean Section

Секции спрес-файла

- `%prep` - команды для подготовки пакета к команде `make` (исправления в исходниках, установка доп. пакетов). Это shell скрипт. НО! Есть встроенные макросы для удобства:
 - `%setup` - распаковывает исходные тексты и делает `cd` к ним
 - `%patch` - накладывает патчи автоматически
 - макросы - записываются также как секции (запомнить)
- `%build` - Раздел для построения ПО. Макросов нет. Shell скрипт для сборки
- `%install` - установка ПО в операционную систему. Фактически в `./rpmbuild/BUILDROOT/`
 - Подходящие команды для этого раздела `make install`
 - `install, cp`
- `%files` - создается вручную. Здесь перечислены файлы которые попадут в пакет

Сборка RPM

- `cd ~/rpmbuild`
- `rpmbuild -bb SPECS/nginx.spec # собрать RPM`
- `rpmbuild -bs SPECS/nginx.spec # собрать SRPM`
- `rpmbuild -ba SPECS/nginx.spec # собрать RPM и SRPM`
- `rpmbuild -bc SPECS/nginx.spec # только %build`
- `rpmbuild -bi SPECS/nginx.spec # только %install`

Создание репозитория RPM



Процесс создания репозитория

- Собираем пакеты
- Копируем *.rpm в директорию
- Инициализируем репозиторий (createrepo)
- Формируем отдачу по HTTP
- Прописываем в конфиге путь к нему

Сборка DEV-пакетов



Процесс сборки пакетов

- Модифицируем файл управления пакетом control файл
 - `c_plusplus_example/myapp/DEBIAN/control`
- Пересборка пакета
 - `dpkg-deb --build ./myapp`
- Копируем пакеты в репозиторий
 - `cp myapp.deb ../repo/`
- Создание индекса пакетов (Packages.gz)
 - `pkg-scandebfiles . /dev/null | gzip -9c > Packages.gz`
- Проверка доступности пакета
 - `apt-cache policy myapp`
- Установка обновленного пакета
 - `sudo apt install myapp`

Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Практика

Домашнее задание

Методичка:

<https://drive.google.com/file/d/1v6dwg8TTabxQNMQadhqVBxetT8gYCsa/view?usp=sharing>

1. Создать свой RPM (можно взять свое приложение, либо собрать Apache с определенными опциями)
2. Создать свой репо и разместить там свой RPM
3. Реализовать это все либо в Vagrant, либо развернуть у себя через Nginx и дать ссылку на репо

В чат ДЗ отправьте ссылку на ваш git-репозиторий.



Сроки выполнения: указаны в личном кабинете



Список материалов для изучения

1. <https://wiki.debian.org/DebianRepository/Format>
2. <https://ubuntu.com/server/docs/package-management>
3. <https://wiki.archlinux.org/title/Pacman/Rosetta>
4. <https://wiki.debian.org/RPM>
5. <https://rpm.org/documentation.html>
6. <https://habr.com/ru/articles/301292/>
7. <https://debian-handbook.info/browse/ru-RU/stable/sect.source-package-structure.html>
8. <https://habr.com/ru/articles/282217/>
9. <https://snapcraft.io/docs/installing-snap-on-ubuntu>
10. <https://girirajsharma.wordpress.com/2015/09/27/build-your-own-custom-nginx/>
11. https://github.com/google/ngx_brotli
12. <https://github.com/nixuser/package-example/blob/main/shellspec.spec>



Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Рефлексия

Цели вебинара

Проверка достижения целей

1. Понимать принципы управления пакетами
2. Узнать процесс сборки пакетов

Рефлексия



Что было самым полезным на занятии?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!

Приходите на следующие вебинары



Лавлинский Николай

Технический директор «Метод Лаб»

https://t.me/methodlab_tg

<https://www.methodlab.ru/>

<https://www.youtube.com/c/NickLavlinsky>

https://www.youtube.com/@site_support

<https://vk.com/nick.lavlinsky>

